

Lecture 2: Haskell

1

Introduction to Functional Programming Haskell

- More at <http://www.haskell.org/>
- Named after Haskell B. Curry, one of the pioneers of lambda calculus
- First defined in 1990
- We will be using the **Glasgow Haskell Compiler interactive** GHCi (2010) interface, which is
 - freely available
 - Efficient and compact
 - Has a flexible user interface
- Haskell scripts use the **.hs** extension
- Haskell is a general purpose, purely functional programming language

2

Introduction to Functional Programming Haskell

- Haskell is a widely used purely functional language with the following features:

✧ Functional Language

- In conventional programming language, we instruct the compiler a series of tasks which is nothing but telling your computer "what to do" and "how to do?" But in Haskell we will tell our computer "what it is?"

✧ Modularity

- A Haskell application is nothing but a series of **functions**
- We can say that a Haskell application is a collection of numerous small Haskell applications

3

Introduction to Functional Programming Haskell

- Haskell is a widely used purely functional language with the following features:

✧ Laziness

- Haskell is a **lazy language** meaning that Haskell won't evaluate **any expression** without any reason
- When the evaluation engine finds that an expression needs to be evaluated, then it creates a **thunk data structure** to collect all the required information for that specific evaluation and a pointer to that **thunk data structure**
- The evaluation engine will start working only when it is required to evaluate that specific expression

4

Introduction to Functional Programming Haskell

✧ Statically Typed

- In conventional programming language, we need to define a series of variables along with their type. In contrast, Haskell is a type inference language
- By the term, type inference language, we mean the Haskell compiler is intelligent enough to **figure out the type of the variable declared**, hence we need not explicitly mention the type of the variable used

✧ Maintainability

- Haskell applications are modular and hence, it is very easy and cost-effective to maintain them

5

Data Types

6

Introduction to Functional Programming

Data Types

❖ Definitions

- A definition associates a **name/identifier** with a value of a particular **type** using **name::Type**
 - Names/identifiers should begin with a small letter and types with a capital letter
 - the **::** means "is a/an"
- ```
age :: Integer
 ◦ associates the name age with an Integer type
 ◦ age is an Integer

square :: Integer -> Integer
square n = n*n
 ◦ the -> indicates that square is a function
 ◦ the input to square function is an Integer, and the output is also an Integer
```

7

## Introduction to Functional Programming

## Data Types

- To summarise the important points made so far:
  - Scripts are **collections of definitions** supplied by the programmer.
  - Definitions are **expressed as equations** between certain kinds of expression and describe mathematical functions.
  - During a session, **expressions are submitted for evaluation**; these expressions can contain references to the functions defined in the script

8

## Haskell Programming Language

## Data Types

- In the notation we are going to describe, the universe of values is partitioned into organised collections, called **types**
- A type is a collection of values grouped together because they are the same sort of thing, even though they are different
- Types can be divided into **two kinds**
- Firstly, there are **basic types** whose values are given as **primitive**
- For example numbers constitute a basic type (the type *num*), as do truth-values the type *bool* and characters (the type *char*)
- Secondly, there are **compound (or derived)** types, whose values are constructed from those of other

9

## Haskell Programming Language

## Data Types

- There are two stages of analysis when an expression is submitted for evaluation
- ❖ **Syntax analysis**
  - The first stage
  - The expression is first checked to see whether it conforms to the correct syntax laid down for expressions
  - If it does not, the computer signals a **syntax error**
  - This stage is called **syntax-analysis**

10

## Haskell Programming Language

## Data Types

- There are two stages of analysis when an expression is submitted for evaluation
- ❖ **Type analysis**
  - Second stage
  - Performed when the expression conforms to the syntax
  - If the expression does, then the expression is analyzed to see if it **possesses a sensible type**
  - This stage is called **type-analysis**
  - If the expression fails to pass this stage, the computer signals a **type error**

11

## Haskell Programming Language

## Data Types

- There are two stages of analysis when an expression is submitted for evaluation
- Only if the expression passes both the syntax analysis and type analysis stages can the process of evaluation begin
- Similar remarks apply to definitions before a script is accepted
- **Strong typing** is important because adherence to the discipline can help in the design of clear and well-structured programs
- When declaring the variable, you must also specify the variable type
- What is more, a wide range of logical errors can be trapped by any computer which enforces it

12

## Haskell Programming Language

## Data Types

♦ **Integers**: `Int` and `Integer` type

- The Haskell `Integer` contains whole numbers, positive numbers, negative numbers, and 0

e.g.    0  
          76  
         -890  
        921425

- `Int` type generally of 2 bytes
- Haskell `Integer` type is large
- `Int` is a type class representing the `Integer` types data
- Every whole number within the range of 2147483647 to -2147483647 comes under the `Int` type class
- `Integer` can be considered as a superset of `Int`, this value is not bounded by any number, hence an `Integer` can be of any length without any limitation

13

## Haskell Programming Language

## Data Types

- Arithmetic** on integers is done using operations: `-`, `*`, `/`, `+`, `div`, `mod`, `abs`, `negate`, `^`,

- `mod` – gives the remainder from a whole number division e.g. `mod 14 3 = 2`
- `div` – gives the whole number division e.g. `div 14 3 = 3`
- `Negate` – change the sign of an integer

- Relational operators** take two integers and return a boolean `Bool` type `True` or `False`

- These are `>`, `<`, `>=`, `=`, `/=`, `<=`,
- E.g. `equal:: Integer -> Integer -> Bool`

14

## Haskell Programming Language

## Data Types

♦ **The Booleans**: `Bool` type

- This type is used for comparisons
- In Haskell it is called `Bool` and contains only two values `T` or `F`
- The operators provided are

`&&`    `and`  
`||`    `or`  
`not`   `not`

- Booleans can be the arguments to, or the results of functions
- Booleans values can also be compared for equality or inequality using the `==` and `/=` operators

```
Prelude> True && True
True
Prelude> True && False
False
Prelude> True || True
True
Prelude> True || False
True
```

15

## Haskell Programming Language

## Data Types

♦ **Floating point numbers**: `Float` type

- Division of integers gives non-integer result, that is, numbers with fractions
- Haskell uses fixed amount of space to store floats, meaning that not all fractions can be represented as floating point numbers
- This can affect the accuracy of floating point numbers
- The decimal point positions is not the same for all numbers
- Scientific notation representation of numbers is allowed in Haskell, which is more convenient
- E.g.  $1.343e7 = 1.343 \times 10^7 = 1\ 3430\ 000$   
 $-3.412e03 = -3.412 \times 10^3 = 3412$   
 $567.8e-2 = 567.8 \times 10^{-2} = 5.678$

16

## Haskell Programming Language

## Data Types

♦ **Floating point numbers**: `Float` type

- A range of operators and functions are provided
- Basic arithmetic `+`, `-`, `*`, `/`, `^`, `**`
- Equality and ordering operations `==`, `/=`, `<`, `>`, `<=`, `>=`

17

## Haskell Programming Language

## Data Types

♦ **Characters**: `Char` type

- These are letters, digits, special characters, Expressions such as `'a'`, `3` are literals
- In Haskell, this is the `Char` type
- Literal characters are enclosed in single quotes e.g. `'y'`, `'6'`
- Some special characters are:

```
'\t' tab

'\n' new line

'\' ' backslash

'\'' single quotes

'\"' ' double quotes
```

18

## Haskell Programming Language

## Data Types

## ♦ Characters: Char type

- Have standard ways of coding characters as integers such as ASCII, unicode
- Capital letter A to Z have numbers 65 to 90
- Small letters a to z have codes 97 to 122
  - e.g. '65' and 'A' have the same meaning
- Specify the operation:-
- getChar which gives the character of an integer code
- getCode which gives the integer code of a character

19

## Haskell Programming Language

## Data Types

## ♦ String type :

- A string is nothing but a collection of characters
- Haskell follows the conventional style of representing a string with double quotation
- String is a sequence of characters enclosed in double quotes eg "Tuesday" , "monkey!ndog!ncow"
- Empty string has no characters
- Strings can be joined using ++
- e.g. "cat"++"fish" will print catfish
- How do we print the following?

```
cat
fish
```

20

## Haskell Programming Language

## Data Types

## ♦ String type :

- Some of the string operations:-
- length which gives the string length
- cat which appends one string to the other string
- The show function in Haskell converts from a value to a String
- The read function converts a string to a value

21

## Haskell Programming Language

## Data Types

## ♦ Overloading

- In Haskell, Integers, Int and Booleans can all be compared for equality using the == symbol
- The == symbol is used for equality over any type for which an equality operator can be defined
- This is called overloading
- Also the number 4 and 2 belong to both Integer and Float, they are overloaded

22

## Haskell Programming Language

## Data Types

## ♦ List type

- A list is a sequence of elements of the same type
- The elements enclosed in square brackets
- Lists in Haskell are homogeneous in nature, which means they won't allow you to declare a list of different kind of data type, that is any list like [1,2,3,4,5,a,b,c,d,e,f] will produce an error
- [T] for the type of all lists, whose elements are of type T
- Examples:-
  - [2,4,50] :: [Integer]
  - ['2', '4', '2', 'r'] :: [Char]
  - ["2","int","50"] :: [String]
- length gives the number of elements in a list
- An empty list [] is of length 0

23

## Haskell Programming Language

## Data Types

## ♦ List type

- It is possible to have a list of lists such as:-
  - [['2','S'], ['4','2','r']] :: [[Char]]
  - ["2","int","50"] :: [String]
- Give a simple specification of :-
- length which gives the number of elements in a list
- ++ which concatenates a list to the other list
- occurs which gives the number of occurrences of a given element in a list

24

## Haskell Programming Language

## Data Types

## ❖ Tuple type

- A tuple is a finite sequence of possibly different types
- Components enclosed in round brackets ( )
- A Tuple can be considered as a List, but there are differences in between a Tuple and a List
- A Tuple is an immutable data type, as we cannot modify the number of elements at runtime, whereas a List is a mutable data type
- List is a homogeneous data type, but Tuple is heterogeneous in nature, because a Tuple may contain different type of data inside it.
- Tuples are represented by single parenthesis, lists by []

25

## Haskell Programming Language

## Data Types

## ❖ Tuple type

- The type of a tuple gives its arity
- Tuples must have a finite arity so that the tuple type can be calculated before evaluation
- No restrictions on the types of the components of a tuple, may include:-
  - Tuples of tuples
  - Tuples of lists
  - Lists of tuples
- Examples:-
 

```
(50, ("cs", False)) :: (Integer, (String, Bool))
(['2', 'a', 'x'], [False, False, False]) :: ([Char], [Bool])
[("cs", False), ("sc", True)] :: [(String, Bool)]
```

27

## Haskell Programming Language

## Data Types

## ❖ Function's type

- A function takes arguments and returns the results
- Each argument has a type, results also
- Examples:-
 

```
add :: (Integer, Integer) -> Integer
isIn :: (String, [String]) -> Bool
add :: (String, [String]) -> [String]
head :: [Integer] -> Integer
fst :: (Integer, String) -> Integer
```
- Give a simple specification of :-
  - tail which gives the tail of list
  - snd which gives the second element of a pair

28

## Functions

29

## Introduction to Functional Programming

## Functions

- A function transforms input into output
- Functions defined in a program are used to write expressions to be evaluated
- In the context of functional programming, a function can be thought of as equivalent to a mathematical function, i.e., a black box mapping a given input to a unique output



- The function gives an output value which depends upon the input values(s)
- The exact syntax of how to write a functions depends on the language that you are programming in

36

## Introduction to Functional Programming

## Functions

- Functions can make code more efficient, easier to read, and elegant
- A function can be called and reused several times
- Information can be passed to a function and the function can send information back
- Many programming languages have built-in functions that you can access in their library, but you can also create your own functions

37

## Introduction to Functional Programming Functions

### ❖ Void Functions

- A void function means that the function does not return a value
- Void functions are used to complete a set of instructions
- They are static and don't change and they always complete the same instructions
- e.g., the function `helloWorld` which outputs "Hello World"

38

## Introduction to Functional Programming Functions

### ❖ Functions That Require Values

- These are functions that need some data to be executed
- In the brackets, we can declare that data is needed to run our function
- Then we can use the data passed to our function in the function

### ❖ Functions That Return a Value

- These functions act on some data and return result
- This is particularly valuable when you want to alter data before you use it.
- For example, a function that requires two integers and will return the sum as an integer

39

## Introduction to Functional Programming Functions

- An important feature of functional programming is that functions are treated no differently from primitive values such as boolean values and integers
- A function
  - can be stored in a variable
  - can also be passed as an actual argument to another function
  - or even returned as a return value of another function
- Such values are called **first-class objects** because they are the most basic element comprising a program
- Hence functions are first-class objects in functional languages

40

## Introduction to Functional Programming Functions

### • Remember, FP functions are Pure functions

- They always produce the same output for same arguments irrespective of anything else.
- Also, they have no side-effects i.e. they do not modify any arguments or local/global variables or input/output streams
- Later property is called **immutability** – the pure function's only result is the value it returns
- This means that programs done using functional programming are easy to debug because pure functions have no side effects or hidden I/O
- Pure functions also make it easier to write parallel/concurrent applications
- When the code is written in this style, a smart compiler can do many things – it can parallelize the instructions, wait to evaluate results when needing them, and memorize the results since the results never change as long as the input doesn't change.

41

## Introduction to Functional Programming Functions

- To define a function

```
square :: Integer -> Integer
square n = n * n
```

- The first line declares the function called `square`, taking an integer type to an integer type
- **Names/identifiers** should begin with a small letter and **types** with a capital letter
- The `::` means "is a/an"
- `n` is an unknown or a variable
- The **arrow** signifies that it is a function
- The **input**, of type integer, appears before the arrow
- The **output** is also of type integer, appearing after the arrow
- To evaluate the function `square` applied to an expression `y`, it becomes `square y = y * y`

42

## Introduction to Functional Programming Functions

- The variables in a function stand for arbitrary values
- These variables are called **formal parameters** of the function
- They stand for arbitrary values of the parameters
- The **actual parameters** are supplied when the function is applied
- Parameters are also called parameters
- A formal argument/parameter serves only as a placeholder for an actual parameter

43